Finite State Machines

Contemporary Digital Electronic Systems — Chapter 13

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. FSM Modeling
3. FSM Design
4. Level-to-Pulse Converter
5. Bit Sequence Detector
6. Practical Aspects
7. Summary

Why This Chapter Matters

A counter walks through its states in a fixed order. A finite-state machine chooses its next state from the current inputs: a counter with branches. FSMs describe traffic lights, serial protocols, button debouncers, vending machines and processor control units.

After this chapter, you can:

1. Draw a state diagram and state table from a word description.
2. Distinguish Moore and Mealy machines.
3. Assign binary codes to states (binary, Gray, one-hot) and explain the trade-offs.
4. Derive next-state and output equations from a state table.
5. Implement a small FSM with gates and flip-flops.

© Muním Zabidi

An Opening Question

How would you build a circuit that watches a stream of serial bits and asserts an output exactly when it has just seen the pattern `110`?

A plain counter cannot do this. By the end of the chapter you will build the circuit that can.

Table of Contents

1 Why This Chapter

2 **FSM Modeling**

3 FSM Design

4 Level-to-Pulse Converter

5 Bit Sequence Detector

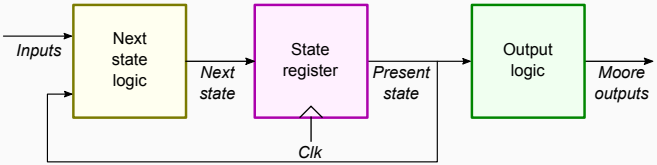
6 Practical Aspects

7 Summary

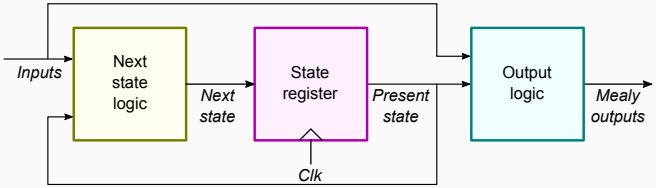
What Is an FSM?

- A digital circuit that follows a predefined sequence of states in response to inputs. Each state holds all the past information needed to predict future behaviour.
- Three parts:
  - State memory: n flip-flops, up to 2^n states, one common (edge-triggered) clock.
  - Next-state logic: present state and inputs give the next state.
  - Output logic: produces the outputs.
- Used for decision-making logic, from simple controls to microprocessors.

Moore and Mealy Machines



Moore

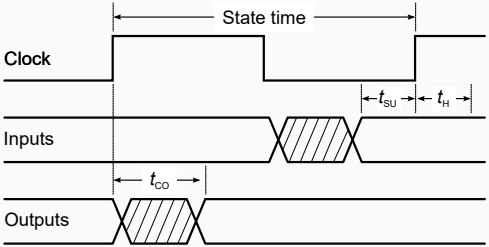


Mealy

- Moore: output depends only on the *state*. Inputs are sampled only at clock events: easy to analyse, but often needs more states.
- Mealy: output depends on state *and input*, so it changes as soon as the input does: a cycle faster, fewer states, but possible glitches.

© Muním Zabidi

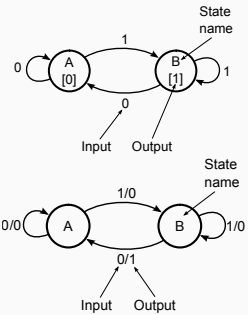
State Machine Timing



- The state time is the interval between clocking events.
- At the edge, the present state and inputs are sampled; the next state and outputs are computed. New outputs are stable after t_{CO}.
- Inputs and next state must be stable for t_{SU} before and t_H after the edge. These set the FSM's maximum speed.

© Muním Zabidi

State Diagrams



Moore Mealy

- States: circles, one per state time.
- State names: symbolic (s_0, *idle*, *ready*).
- Arrows: transitions, labelled with input values; a state has an exit for every input combination: 2^n arrows for n inputs. For larger n, arrows carry Boolean conditions (Chapter 15).
- Outputs: Moore, inside the state; Mealy, after a "/" on the arrow.

© Muním Zabidi

State Table

Present state	Input X	Next state	Output Y
A	0	A	0
A	1	B	
B	0	A	1
B	1	B	

- A truth table that includes the present state as an input and the next state as an output.
- Level-to-pulse converter, two states: in A with $X = 0$ stay in A; $X = 1$ go to B. In B, $X = 0$ returns to A; $X = 1$ stays in B.

© Muním Zabidi

State Encoding

No	Binary	Gray	One-hot (8)
0	000	000	00000001
1	001	001	00000010
2	010	011	00000100
3	011	010	00001000
4	100	110	00010000
5	101	111	00100000
6	110	101	01000000
7	111	100	10000000

- Binary: $\lceil \log_2 N \rceil$ bits; fewest flip-flops, more decoding logic, several bits may change at once.
- Gray: also $\lceil \log_2 N \rceil$ bits; successive codes differ in one bit; benefit depends on the transition order.
- Johnson: $2k$ states from k bits; one bit changes per step; more flip-flops.
- One-hot: N bits for N states; simple decoding and often simpler logic, more flip-flops.
- Choose by states, speed, hardware and power.

Example: Encoding 6 States

Encoding	Flip-flops	Note
Binary	$\lceil \log_2 6 \rceil = 3$	2 unused patterns
Gray	3	Successive codes differ by one bit
Johnson	$\lceil 6/2 \rceil = 3$	$2k = 6$ states: 000, 001, 011, 111, 110, 100; no unused patterns
One-hot	6	Each state is a single flip-flop

- Binary, Gray and Johnson all need 3 flip-flops but are not interchangeable.
- One-hot uses twice as many flip-flops, but reading a state or an output is a direct read of one flip-flop.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 FSM Modeling

3 **FSM Design**

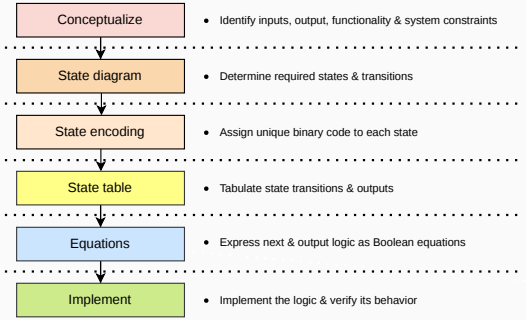
4 Level-to-Pulse Converter

5 Bit Sequence Detector

6 Practical Aspects

7 Summary

The FSM Design Flow



- Identify inputs, outputs and constraints; draw the state diagram; encode the states; build the state table; derive next-state and output equations; implement and verify.
- Like combinational design, plus state assignment.
- The first two steps need design judgement. When an FSM misbehaves, check the specification and state diagram first.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 FSM Modeling

3 FSM Design

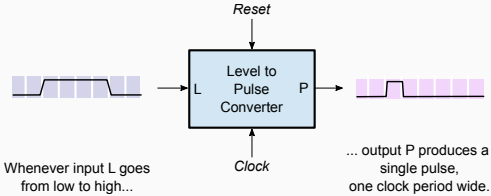
4 **Level-to-Pulse Converter**

5 Bit Sequence Detector

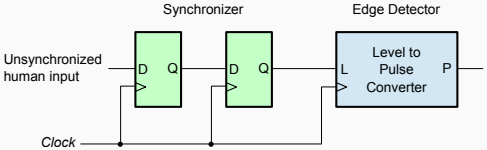
6 Practical Aspects

7 Summary

Level-to-Pulse Converter

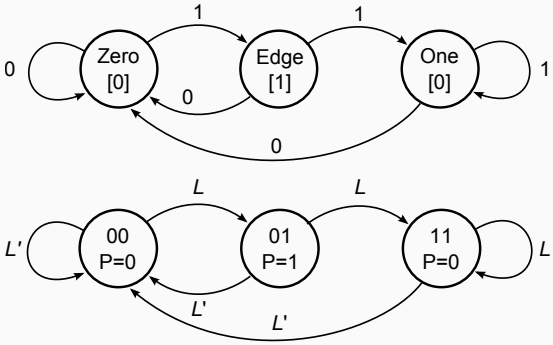


- Outputs a single-cycle pulse each time the input goes high, however long it stays high.
- An edge detector: a button press lasts many cycles, but a counter needs one clean enable.



- Human inputs are asynchronous: pass them through two series D flip-flops (a synchroniser), or the edge may fall inside setup/hold, the circuit captures an unpredictable value and fails.

Moore Version

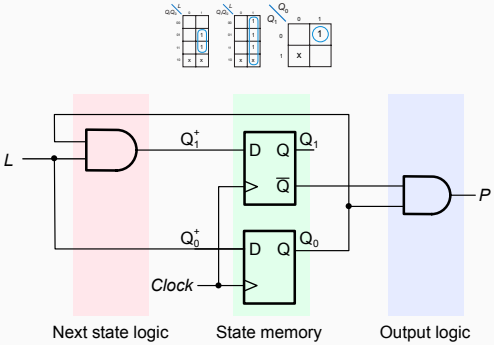


- Three states, encoded 00 (waiting), 01 (edge detected, $P = 1$), 11 (waiting for falling edge).
- The lower diagram spells out each transition condition: it scales to many inputs and outputs.

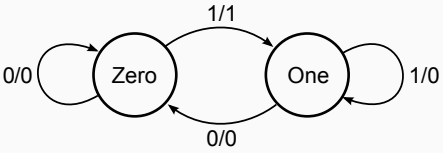
Moore: State Table and Equations

Q_1	Q_0	L	Q_1^+	Q_0^+	P
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	1
1	1	0	0	0	0
1	1	1	1	1	0

$$Q_1^+ = LQ_0 \qquad Q_0^+ = L \qquad P = \overline{Q_1}Q_0$$

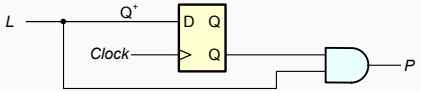


Mealy Version: Two States



Q	L	Q^+	P
0	0	0	0
0	1	1	1
1	0	0	0
1	1	1	0

- A Mealy output reacts to the input immediately, so no extra state is needed to hold the pulse.
- By inspection (no K-map): $Q^+ = L, P = \overline{Q}L$.



- One fewer state bit and flip-flop, simpler next-state logic.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 FSM Modeling

3 FSM Design

4 Level-to-Pulse Converter

5 **Bit Sequence Detector**

6 Practical Aspects

7 Summary

1. Define the sequence to detect; the output is active when it is fully recognised in the last state.
2. Draw an initial state diagram for the ideal path: the input is exactly the sequence from the start.
3. Complete it: every state needs an exit for input 0 and one for input 1.

The 110 detector

Assert z for one clock cycle whenever `110` appears, including overlapping occurrences.

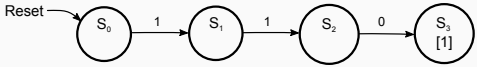
```
x:  1 1 0 0 0 1 1 0 1 1 1 1 0 0
z:  0 0 1 0 0 0 0 1 0 0 0 0 1 0
```

© Muním Zabidi

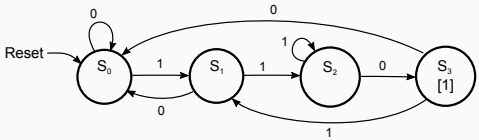
110 Detector: Moore Design

State	Meaning
S_0	No matching bits
S_1	First 1 found
S_2	Second 1 found
S_3	0 found: assert z

- One state per bit of progress plus an idle state: four states.



Ideal path



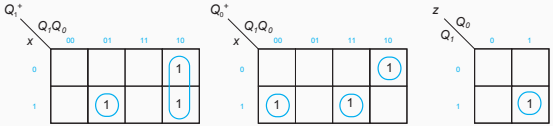
Completed diagram

© Munim Zabidi

Moore Detector: Table and Equations

Q_1	Q_0	x	Q_1^+	Q_0^+	z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	1	1	0
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	0	1	1

$S_0 = 00,\ S_1 = 01,\ S_2 = 10,\ S_3 = 11$

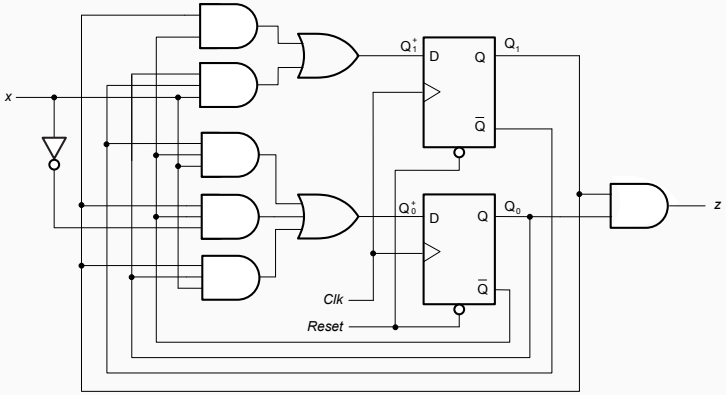


$$Q_1^+ = Q_1\overline{Q_0} + \overline{Q_1}Q_0x$$

$$Q_0^+ = \overline{Q_1}\,\overline{Q_0}x + Q_1\overline{Q_0}\,\overline{x} + Q_1Q_0x$$

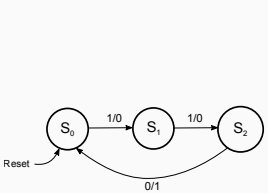
$$z = Q_1Q_0$$

Moore Detector: Schematic

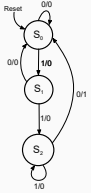


110 Detector: Mealy Design

- Three states, not four: the output is asserted on the transition from S_2 back to S_0 (the 0 that completes `110`).



Ideal path

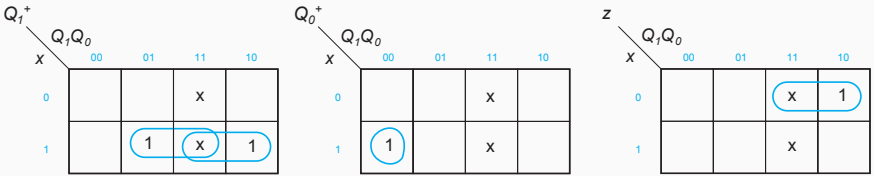


Completed

Q_1	Q_0	x	Q_1^+	Q_0^+	z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	X	X	X	X

Code 11 is unused: don't cares.

Mealy Detector: Equations



Using the don't cares

$Q_1^+ = Q_1x + Q_0x \qquad Q_0^+ = \overline{Q_1}\,\overline{Q_0}x \qquad z = Q_1\overline{x}$

Implementing the circuit is left as an exercise, following the Moore pattern.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 FSM Modeling

3 FSM Design

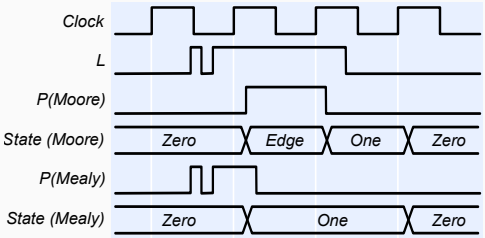
4 Level-to-Pulse Converter

5 Bit Sequence Detector

6 **Practical Aspects**

7 Summary

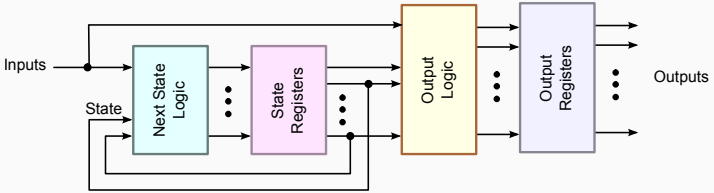
Moore versus Mealy in Practice



- Mealy: output $= f$(state, inputs). Moore: output $= f$(state).
- The input L has a glitch between clock edges. The Moore machine looks only at values sampled at clock edges, so it ignores the glitch.
- The Mealy output follows L between edges, so the glitch can reach the output. State transitions still use the value sampled at the edge.

© Muním Zabidi

Registered (Synchronous) Mealy Outputs



- Moore outputs are easier to reason about. Mealy outputs respond faster and often need fewer states, but may glitch.
- Mealy outputs can upset other circuits or state machines they drive. Synchronous controls help but do not remove every timing concern.
- Registering the outputs keeps them stable between clock edges, at the cost of one clock cycle of delay.

© Muním Zabidi

The 110 detector

A plain counter walks a fixed sequence and cannot react to the data. An FSM can: one state per bit of progress (S_0 idle, S_1 one 1, S_2 two 1s), moving on a matching bit and falling back on a mismatch. In the Moore version a fourth state S_3 asserts z; in the Mealy version z is asserted on the transition out of S_2 when $x = 0$.

Table of Contents

1. Why This Chapter
2. FSM Modeling
3. FSM Design
4. Level-to-Pulse Converter
5. Bit Sequence Detector
6. Practical Aspects
7. **Summary**

Summary

- A Moore machine's output depends only on the current state: stable between clock edges.
- A Mealy machine's output depends on state and input: it can respond within a cycle but may glitch.
- State encoding (binary, Gray, one-hot) trades flip-flop count against logic complexity and speed.
- Design flow: state diagram → state table → state encoding → next-state and output equations → schematic.
- A state diagram is a specification of the machine's behaviour.

Looking ahead

Chapter 14 reverses the direction: given a circuit, derive its state diagram. Chapter 15 extends design to controllers with several inputs and outputs.